Custom HashMap - How it works

Home notes + DESIGN.md + source code | compact print

In one line - Keys go into buckets by hash. Collisions chain in a list. Too full? Grow and rehash.

Story - how this LLD works

Like a set of drawers. hash(key) % number_of_drawers picks the drawer. Inside the drawer, entries are a linked list (separate chaining). Each drawer starts with a dummy sentinel node so insert/remove is simpler. When entries > 0.75 * drawers, double drawers and redistribute everything.

Who does what

CustomHashMap: put / get / remove. Owns bucket array and count.
Node: One entry: key, value, next, prev (doubly linked).
Sentinel head: Dummy node at start of each bucket chain.
Load factor 0.75: Trigger for growing the table.

Main pieces

Bucket array

- index = hash % n
- each has sentinel

Chain

- Node <-> Node
- key, value

put

- find or insert
- maybe rehash

get / remove

- walk chain
- unlink

Step-by-step at runtime

1. put('John', 78): hash -> bucket index.
2. Walk chain; if key exists, update value.
3. Else insert new node after sentinel; count++.
4. If count too high vs capacity -> rehash to 2x buckets.
5. get('John'): same hash, walk chain, return value or None.
6. remove: unlink node from its neighbors, count--.

Why this design (plain words)

Why sentinel? - Fewer special cases for empty bucket / remove head.

Why doubly linked? - Once you found the node, unlinking is O(1).

Why rehash? - Keep average chain short so get/put stay fast.

Remember

- Average O(1); worst O(n) if everything collides.
- Python hash() is not stable across processes.
- remove does not shrink the table in this demo.

OOP in this project

Encapsulation - You only call put/get/remove - buckets stay private.

Abstraction - HashMap ADT hides collision handling.

Inheritance - Not the main tool here - composition of nodes.

Polymorphism - Works for any hashable key type via generics.

DESIGN.md - full file

Complete markdown from lld/hashmap/DESIGN.md (Q&A for learning).

Full DESIGN.md (same file as in this folder) - DESIGN.md

Designing a Custom HashMap - LLD Q&A

Run: python -m lld.hashmap.main
Focus: Data structure internals (not classic GoF patterns)

Q1. What problem are we solving?

A. Implement a hash map from scratch with:
Generic key-value storage (CustomHashMap[K, V])
Separate chaining for collisions
Dynamic resizing when load factor is exceeded
get / put / remove with average O(1)
Similar spirit to Java HashMap bucket chains.

Q2. What are the core pieces?

- Class - Responsibility
- Node[K, V] - Doubly-linked list node: key, val, next, prev
- CustomHashMap[K, V] - Bucket array, chaining, rehash

Q3. How is the package structured?

```
hashmap/
|-- custom_hash_map.py  # Node + CustomHashMap
|-- main.py             # Demo put/get
```

Simple package - pure data-structure LLD, no service/strategy layers.

Q4. How does internal design work?

- Concern - Approach
- Buckets - Array of chains; each bucket has a sentinel head
- Hash - hash(key) % bucket_count
- Load factor - 0.75 - rehash when count > 0.75 * capacity
- Rehash - Double capacity (cap 1 << 30), redistribute nodes
- Collisions - Separate chaining via doubly-linked lists
- Sentinel heads avoid null edge cases on insert/remove.

Q5. How do core operations work?

put (amortized O(1))

```
find node by key in bucket
  -> exists: update value
  -> else: insert after sentinel, count++
  -> if over load factor: rehash (2x)
```

get (average O(1)) - traverse bucket chain via find_node.
remove (average O(1)) - unlink from doubly-linked list, decrement count.

Q6. What does the demo show?

```
put several entries -> get existing key -> get missing key (None)
```

Initial capacity is small (4) for demo; rehash grows as load increases.

Q7. How do you extend this design?

- Add... - How
- Custom hash/eq - Accept hash_fn / eq_fn callables
- Treeify long chains - Convert chain -> tree when length > 8 (Java 8 style)
- Iteration - keys(), values(), items()
- Concurrent map - Striping or per-bucket locks
- Shrink on delete - Rehash down when load drops

Q8. Common interview follow-ups

Q. Why doubly-linked lists in buckets?
O(1) unlink on remove once the node is found; sentinel simplifies head/tail cases.
Q. What happens at load factor 0.75?
Rehash doubles buckets and redistributes - keeps average chain length low.
Q. Is Python hash() stable?
Not across processes (hash randomization) - important if you serialize hash codes.
Q. Does remove shrink the table?
No - only grows on put. Mention shrink as a production enhancement.
Q. Open addressing vs chaining?

This design uses chaining. Open addressing stores entries in the array itself (linear/quadratic probing) - discuss trade-offs if asked.

Source code - lld/hashmap/

3 Python files below (compact font to save pages). Read with the notes: find the class name from the cast, then open that file here.
- Tip: start at main.py, then service/, then model/ + strategy/.

FILE: main.py (18 lines)

```
 1| from .custom_hash_map import CustomHashMap
 2|
 3|
 4| def main() -> None:
 5|     map_: CustomHashMap[str, int] = CustomHashMap()
 6|     map_.put("Shubh", 90)
 7|     map_.put("Karan", 80)
 8|     map_.put("Alice", 85)
 9|     map_.put("John", 78)
10|     map_.put("Tom", 82)
11|     map_.put("Parth", 95)
12|
13|     print(map_.get("John"))
14|     print(map_.get("Bob"))
15|
16|
17| if __name__ == "__main__":
18|     main()
```

FILE: custom_hash_map.py (110 lines)

```
 1| from dataclasses import dataclass
 2| from typing import Generic, Optional, TypeVar
 3|
 4| K = TypeVar("K")
 5| V = TypeVar("V")
 6|
 7|
 8| @dataclass
 9| class Node(Generic[K, V]):
10|     key: Optional[K]
11|     val: Optional[V]
12|     next: Optional["Node[K, V]"] = None
13|     prev: Optional["Node[K, V]"] = None
14|
15|
16| class CustomHashMap(Generic[K, V]):
17|     def __init__(self) -> None:
18|         self._INITIAL_SIZE = 4
19|         self._MAX_CAPACITY = 1 << 30
20|         self._LOAD_FACTOR = 0.75
21|         self._count_of_nodes = 0
22|         self._map: list[Node[K, V]] = [None] * self._INITIAL_SIZE  # type: ignore[list-item]
23|         for i in range(self._INITIAL_SIZE):
24|             self._map[i] = Node(None, None)
25|             self._map[i].next = Node(None, None)
26|             self._map[i].next.prev = self._map[i]
27|
28|     def get(self, key: K) -> Optional[V]:
29|         node = self.find_node(key)
30|         return None if node is None else node.val
31|
32|     def put(self, key: K, val: V) -> None:
33|         node = self.find_node(key)
34|         if node is not None:
35|             node.val = val
36|             return
37|
38|         bucket = hash(key) % len(self._map)
39|         head = self._map[bucket]
40|
41|         new_node = Node(key, val)
42|         old_first = head.next
43|         head.next = new_node
44|         new_node.prev = head
45|         new_node.next = old_first
46|         old_first.prev = new_node
47|
48|         self._count_of_nodes += 1
49|
50|         if self._count_of_nodes > self._LOAD_FACTOR * len(self._map):
51|             self._rehash(len(self._map) * 2)
52|
53|     def remove(self, key: K) -> None:
54|         node_to_remove = self.find_node(key)
55|
56|         if node_to_remove is None:
57|             return
58|
59|         prev_node = node_to_remove.prev
60|         next_node = node_to_remove.next
61|
62|         prev_node.next = next_node
63|         next_node.prev = prev_node
64|
65|         self._count_of_nodes -= 1
66|
67|     def get_size(self) -> int:
68|         return self._count_of_nodes
69|
70|     def _rehash(self, new_size: int) -> None:
71|         if new_size > self._MAX_CAPACITY:
72|             print("Hashmap is exceeding max capacity")
```

```
            return

        new_map: list[Node[K, V]] = [None] * new_size  # type: ignore[list-item]
        for i in range(new_size):
            new_map[i] = Node(None, None)
            new_map[i].next = Node(None, None)
            new_map[i].next.prev = new_map[i]

        for curr in self._map:
            while curr is not None:
                if curr.key is None:
                    curr = curr.next
                    continue

                new_bucket = hash(curr.key) % new_size
                new_head = new_map[new_bucket]
                old_first = new_head.next

                next_node = curr.next

                new_head.next = curr
                curr.prev = new_head
                curr.next = old_first
                old_first.prev = curr

                curr = next_node

        self._map = new_map

    def find_node(self, key: K) -> Optional[Node[K, V]]:
        bucket = hash(key) % len(self._map)
        head = self._map[bucket]

        while head is not None:
            if head.key is not None and head.key == key:
                return head
            head = head.next
        return None
```

FILE: __init__.py (3 lines)

```
from .custom_hash_map import CustomHashMap

__all__ = ["CustomHashMap"]
```